04_modeling_michael

June 13, 2026

1 SUPPORT2 research questions: disease group, day-3 physiology, and the ML questions

This notebook covers five questions on the Gold feature set: two inference questions (disease group, day-3 physiology) and three prediction questions (a day-3 classifier, whether it beats the 1990s surv6m score, and whether the same variables drive both the inference and the prediction). Every section ran on Gold. The one exception is ML-Q2, where the legacy surv6m score was read from the Silver table for the comparison, since surv6m is a model output and was not part of the Gold features.

2 RQ1: disease group and 180-day mortality

[1]:
```
# Imports
import sys
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.formula.api as smf
from statsmodels.stats.multitest import multipletests

# Constants
RANDOM_STATE = 42
ALPHA = 0.05
REFERENCE_GROUP = "ARF/MOSF w/Sepsis"
np.random.seed(RANDOM_STATE)

# add repo root to sys.path so the utils import resolves whether the
# notebook runs from the repo root or from src/notebooks/
_here = Path.cwd()
_proj_root = _here if (_here / "utils").exists() else (_here / "../..").
 ↪resolve()
if str(_proj_root) not in sys.path:
    sys.path.insert(0, str(_proj_root))
from utils.dataset import load_csv  # noqa: E402
```

2.1 RQ1: does a patient's disease group shape their odds of dying within 180 days?

This is the project's first and primary research question, and it sets the baseline the rest of the analysis builds on. The study is organized around disease group, every later model holds it constant, and the disease-group signal is the strongest in the cohort. Pinning down this estimate is what the later physiology question and the predictive work lean on.

As an inference question, the model checks whether a patient's main illness is associated with the odds of death within 180 days, and by how much, after age, sex, and overall severity are held constant. Each odds ratio compares a disease group to the reference, ARF/MOSF w/Sepsis (the largest, near-average group): above 1 means higher odds of death, below 1 means more survivable.

[2]:
```
# Load the Gold feature set (model-ready; it now carries sex, so the inference␣
 ↪runs on it too).
gold = load_csv("support2_model_features.csv")
print("rows, cols:", gold.shape)

# Base rate of the outcome (roughly half the cohort died within 180 days).
base_rate = gold["death_180d"].mean()
print(f"death_180d base rate: {base_rate:.3f} "
      f"({base_rate * 100:.1f}% died within 180 days)")

gold.head()
```

```
rows, cols: (9105, 29)
death_180d base rate: 0.468 (46.8% died within 180 days)
```

[2]:

	age	sex	dzgroup	dzclass	num_co	\
0	62.84998	male	Lung Cancer	Cancer	0	
1	60.33899	female	Cirrhosis	COPD/CHF/Cirrhosis	2	
2	52.74698	female	Cirrhosis	COPD/CHF/Cirrhosis	2	
3	42.38498	female	Lung Cancer	Cancer	2	
4	79.88495	female	ARF/MOSF w/Sepsis	ARF/MOSF	1	

	income	scoma	avtisst	race	diabetes	…	bili	crea	\
0	$11-$25k	0.0	7.000000	other	0	…	0.199982	1.199951	
1	$11-$25k	44.0	29.000000	white	0	…	1.010000	5.500000	
2	under $11k	0.0	13.000000	white	0	…	2.199707	2.000000	
3	under $11k	0.0	7.000000	white	0	…	1.010000	0.799927	
4	under $11k	26.0	18.666656	white	0	…	1.010000	0.799927	

	sod	ph	glucose	bun	urine	sfdm2	adlsc	\
0	141.0	7.459961	135.0	6.51	2502.0	<2 mo. follow-up	7.0	
1	132.0	7.250000	135.0	6.51	2502.0	<2 mo. follow-up	1.0	
2	134.0	7.459961	135.0	6.51	2502.0	<2 mo. follow-up	0.0	
3	139.0	7.419922	135.0	6.51	2502.0	no(M2 and SIP pres)	0.0	
4	143.0	7.509766	135.0	6.51	2502.0	no(M2 and SIP pres)	2.0	

```
   death_180d
0           0
1           1
2           1
3           1
4           0

[5 rows x 29 columns]
```

[3]:
```
# Columns we actually model: the disease group, the controls, and the target.
MODEL_COLS = ["death_180d", "age", "sex", "scoma", "dzgroup"]

# leakage and benchmark columns not in the model
# (picking MODEL_COLS already drops them; this just confirms none slipped␣
 ↪through)
LEAKAGE_COLS = [
    "surv2m", "surv6m", "prg2m", "prg6m", "aps", "sps",
    "death", "hospdead", "d.time", "d_time", "slos",
    "dnr", "dnrday", "sfdm2",
]

model_df = gold[MODEL_COLS].dropna()
print("modeling frame shape:", model_df.shape)

leaked = [c for c in LEAKAGE_COLS if c in model_df.columns]
print("leakage/benchmark columns present:", leaked if leaked else "none␣
 ↪(clean)")

model_df.head()
```

```
modeling frame shape: (9105, 5)
leakage/benchmark columns present: none (clean)
```

[3]:

	death_180d	age	sex	scoma	dzgroup
0	0	62.84998	male	0.0	Lung Cancer
1	1	60.33899	female	44.0	Cirrhosis
2	1	52.74698	female	0.0	Cirrhosis
3	1	42.38498	female	0.0	Lung Cancer
4	0	79.88495	female	26.0	ARF/MOSF w/Sepsis

2.1.1 The model

One multivariable logistic regression was fit:

```
death_180d ~ age + sex + scoma + C(dzgroup, reference = "ARF/MOSF w/Sepsis")
```

- death_180d = died within 180 days (1 / 0), the outcome.
- age = per year, held constant as a confounder.
- sex = a demographic confounder, held constant.

- scoma = the day-3 coma score, the in-model severity control (aps and sps were pulled out as benchmarks, so scoma carries severity here).
- C(dzgroup, …) = disease group as categories, each compared to the sepsis reference.

Logistic because the outcome is yes/no. Multivariable so the disease-group effect is measured after age, sex, and severity are accounted for, not before.

[4]:
```
# Fit the multivariable logistic regression with statsmodels. The reference
# group is fixed so every disease-group odds ratio is read against sepsis.
formula = (
    "death_180d ~ age + sex + scoma + "
    f'C(dzgroup, Treatment(reference="{REFERENCE_GROUP}"))'
)
model = smf.logit(formula, data=model_df).fit(disp=False)
print(model.summary())
```

```
                           Logit Regression Results
==============================================================================
Dep. Variable:              death_180d   No. Observations:                 9105
Model:                           Logit   Df Residuals:                     9094
Method:                            MLE   Df Model:                           10
Date:                 Sat, 13 Jun 2026   Pseudo R-squ.:                 0.1196
Time:                         23:37:37   Log-Likelihood:                -5540.4
converged:                       True   LL-Null:                       -6292.9
Covariance Type:            nonrobust   LLR p-value:                     0.000
==============================================================================
==========================================================
                                                                        coef
std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
------------------------------------------------------
Intercept                                                              -2.0285
0.110    -18.441      0.000      -2.244      -1.813
sex[T.male]                                                             0.0813
0.046      1.755      0.079      -0.010       0.172
C(dzgroup, Treatment(reference="ARF/MOSF w/Sepsis"))[T.CHF]            -0.6157
0.073     -8.381      0.000      -0.760      -0.472
C(dzgroup, Treatment(reference="ARF/MOSF w/Sepsis"))[T.COPD]           -0.4991
0.081     -6.172      0.000      -0.658      -0.341
C(dzgroup, Treatment(reference="ARF/MOSF w/Sepsis"))[T.Cirrhosis]       0.4435
0.100      4.437      0.000       0.248       0.639
C(dzgroup, Treatment(reference="ARF/MOSF w/Sepsis"))[T.Colon Cancer]    0.2639
0.098      2.685      0.007       0.071       0.457
C(dzgroup, Treatment(reference="ARF/MOSF w/Sepsis"))[T.Coma]            0.1903
0.122      1.560      0.119      -0.049       0.430
C(dzgroup, Treatment(reference="ARF/MOSF w/Sepsis"))[T.Lung Cancer]     1.0543
0.080     13.195      0.000       0.898       1.211
C(dzgroup, Treatment(reference="ARF/MOSF w/Sepsis"))[T.MOSF w/Malig]    1.5165
0.097     15.675      0.000       1.327       1.706
```

```
age                                                                    0.0230
0.002     14.692      0.000       0.020       0.026
scoma                                                                  0.0252
0.001     18.580      0.000       0.023       0.028
================================================================================
================================================
```

[5]:
```python
# Build the odds-ratio table from the fitted model.
params = model.params
conf = model.conf_int()
conf.columns = ["ci_low", "ci_high"]

or_table = pd.DataFrame({
    "OR": np.exp(params),
    "OR_ci_low": np.exp(conf["ci_low"]),
    "OR_ci_high": np.exp(conf["ci_high"]),
    "p_value": model.pvalues,
})

# keep just the disease-group terms for the FDR step and the forest plot
dz_mask = or_table.index.str.startswith("C(dzgroup")
dz_table = or_table[dz_mask].copy()

# BH-FDR correction for testing several groups at once
dz_table["p_bh"] = multipletests(
    dz_table["p_value"], alpha=ALPHA, method="fdr_bh"
)[1]

# pull the group name out of the long statsmodels label
dz_table.index = [name.split("T.")[-1].rstrip("]") for name in dz_table.index]
dz_table.index.name = "disease_group"

# sort by odds ratio, highest odds of death at the top
dz_table = dz_table.sort_values("OR", ascending=False)
dz_table[["OR", "OR_ci_low", "OR_ci_high", "p_value", "p_bh"]].round(4)
```

[5]:

disease_group	OR	OR_ci_low	OR_ci_high	p_value	p_bh
MOSF w/Malig	4.5562	3.7692	5.5075	0.0000	0.0000
Lung Cancer	2.8701	2.4540	3.3567	0.0000	0.0000
Cirrhosis	1.5582	1.2810	1.8954	0.0000	0.0000
Colon Cancer	1.3020	1.0739	1.5786	0.0072	0.0085
Coma	1.2097	0.9523	1.5365	0.1189	0.1189
COPD	0.6071	0.5181	0.7113	0.0000	0.0000
CHF	0.5403	0.4678	0.6240	0.0000	0.0000

[6]:
```
# the controls (age, sex, scoma), shown next to the disease groups
control_terms = [t for t in or_table.index
                 if t in ("age", "scoma") or t.startswith("sex")]
or_table.loc[control_terms, ["OR", "OR_ci_low", "OR_ci_high", "p_value"]].
 ↪round(4)
```

[6]:

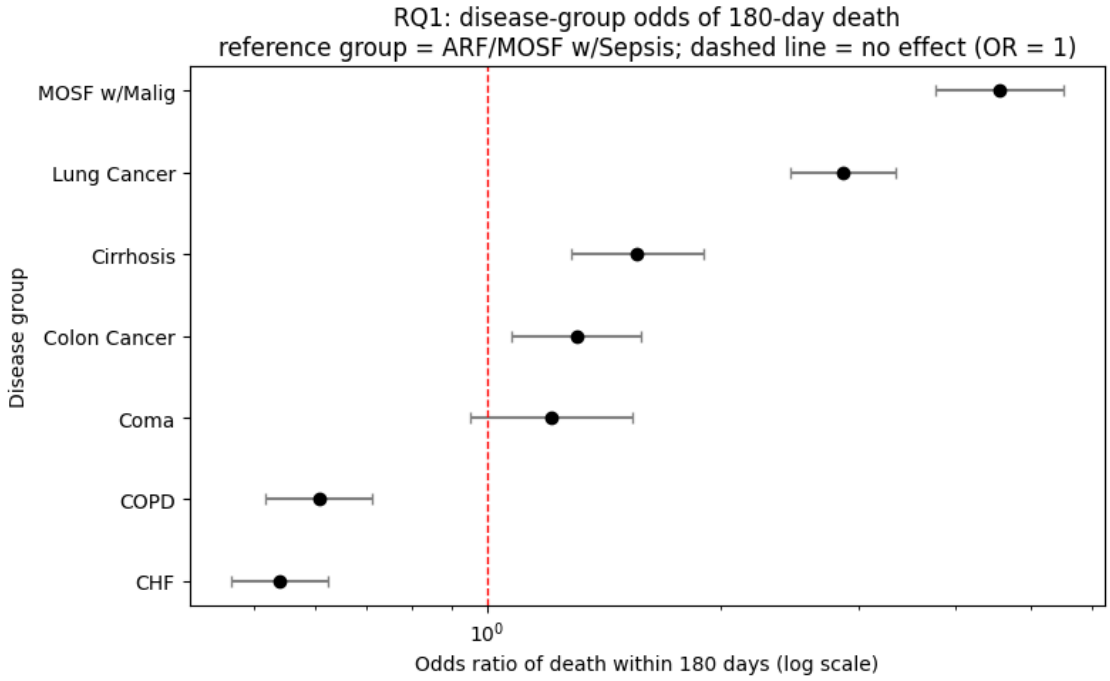
	OR	OR_ci_low	OR_ci_high	p_value
sex[T.male]	1.0846	0.9905	1.1877	0.0793
age	1.0232	1.0201	1.0264	0.0000
scoma	1.0255	1.0228	1.0283	0.0000

[7]:
```
# Labeled forest plot: one dot per disease group with its 95% CI bar.
fig, ax = plt.subplots(figsize=(8, 5))

groups = dz_table.index.tolist()
y = np.arange(len(groups))
or_vals = dz_table["OR"].to_numpy()
low = dz_table["OR_ci_low"].to_numpy()
high = dz_table["OR_ci_high"].to_numpy()

xerr = np.vstack([or_vals - low, high - or_vals])
ax.errorbar(
    or_vals, y, xerr=xerr, fmt="o", color="black",
    ecolor="gray", capsize=3, linestyle="none",
)

ax.axvline(1.0, linestyle="--", color="red", linewidth=1)
ax.set_xscale("log")
ax.set_yticks(y)
ax.set_yticklabels(groups)
ax.invert_yaxis()  # highest odds ratio on top
ax.set_xlabel("Odds ratio of death within 180 days (log scale)")
ax.set_ylabel("Disease group")
ax.set_title(
    "RQ1: disease-group odds of 180-day death\n"
    f"reference group = {REFERENCE_GROUP}; dashed line = no effect (OR = 1)"
)
fig.tight_layout()
plt.show()
```



2.1.2 What the disease groups showed

The disease a patient came in with shaped the odds of dying within 180 days a lot, even after holding age, sex, and severity constant. The highest-odds group was MOSF w/Malig (multi-organ failure with cancer), at about 4.6 times the odds of the sepsis reference group, followed by Lung Cancer at about 2.9 times. Cirrhosis (about 1.6 times) and Colon Cancer (about 1.3 times) were also higher than the reference. On the other end, CHF (heart failure) and COPD were the most survivable, at about 0.54 and 0.61 times the reference odds.

Coma came out at about 1.2 times the reference, but its confidence interval crossed 1 and it did not survive the BH-FDR correction (adjusted $p = 0.12$), so this model cannot tell coma apart from the sepsis reference. That runs against an earlier expectation that coma would come in high.

The two controls behaved as expected. Each extra year of age multiplied the odds by about 1.02 (a 2.3 percent rise per year, $p < 0.001$), and each point of the coma score raised the odds by about 1.03 ($p < 0.001$). Sex made only a small difference: men had about 1.08 times the odds of women, and that gap was not significant ($p = 0.08$) once disease, age, and severity were accounted for. That is why sex is in the model as a control, not as one of the disease-group effects the question is actually about: its own effect is small, but adjusting for it keeps the disease-group odds ratios comparable.

Every disease-group difference except Coma stayed significant after the Benjamini-Hochberg FDR correction, so these are not an artifact of testing several groups at once.

3 RQ2: day-3 physiology beyond disease group

[8]:
```
# Imports
import sys
from pathlib import Path

import numpy as np
import pandas as pd
import statsmodels.formula.api as smf
from statsmodels.stats.outliers_influence import variance_inflation_factor
from patsy import dmatrix
from scipy import stats

# Constants
ALPHA = 0.05
REFERENCE_GROUP = "ARF/MOSF w/Sepsis"
CORE_PHYS = ["meanbp", "wblc", "crea"]
SENS_PHYS = ["alb", "bili"]  # heavily imputed with normal values, checked␣
 ↪separately

# add repo root to sys.path so the utils import resolves whether the
# notebook runs from the repo root or from src/notebooks/
_here = Path.cwd()
_proj_root = _here if (_here / "utils").exists() else (_here / "../..").
 ↪resolve()
if str(_proj_root) not in sys.path:
    sys.path.insert(0, str(_proj_root))
from utils.dataset import load_csv  # noqa: E402
```

3.1 RQ2: does day-3 physiology add signal on top of disease group?

Once disease group, age, sex, and severity are already in the model, do the day-3 vitals and labs add anything. The catch is that a sick patient's numbers often just restate how sick the disease already implies, so the real test is whether physiology carries new signal after disease group is accounted for, not on its own.

[9]:
```
gold = load_csv("support2_model_features.csv")
BASE = ["death_180d", "age", "sex", "scoma", "dzgroup"]
model_df = gold[BASE + CORE_PHYS + SENS_PHYS].dropna()
print("rows used:", model_df.shape[0])
print("death_180d base rate:", round(model_df["death_180d"].mean(), 3))
model_df.head()
```

```
rows used: 9105
death_180d base rate: 0.468
```

[9]:

	death_180d	age	sex	scoma	dzgroup	meanbp	wblc	\
0	0	62.84998	male	0.0	Lung Cancer	97.0	6.000000	

```
1           1  60.33899  female   44.0          Cirrhosis    43.0  17.097656
2           1  52.74698  female    0.0          Cirrhosis    70.0   8.500000
3           1  42.38498  female    0.0        Lung Cancer    75.0   9.099609
4           0  79.88495  female   26.0  ARF/MOSF w/Sepsis    59.0  13.500000

       crea       alb      bili
0  1.199951  1.799805  0.199982
1  5.500000  3.500000  1.010000
2  2.000000  3.500000  2.199707
3  0.799927  3.500000  1.010000
4  0.799927  3.500000  1.010000
```

3.1.1 Three nested models

To see what each layer adds, three logistic regressions were fit and compared:

```
M1: death_180d ~ age + sex + scoma
M2: M1 + disease group
M3: M2 + day-3 physiology
```

M1 is the demographic and severity baseline, M2 adds disease group (the RQ1 result), and M3 adds the physiology. Comparing the fit from M1 to M2 to M3 showed how much disease group explained and how much physiology added on top of it. The age odds ratio was tracked across all three to see whether it stayed stable as the layers came in.

[10]:
```
f1 = "death_180d ~ age + sex + scoma"
f2 = f1 + f' + C(dzgroup, Treatment(reference="{REFERENCE_GROUP}"))'
f3 = f2 + " + " + " + ".join(CORE_PHYS + SENS_PHYS)

m1 = smf.logit(f1, model_df).fit(disp=False)
m2 = smf.logit(f2, model_df).fit(disp=False)
m3 = smf.logit(f3, model_df).fit(disp=False)

triple = pd.DataFrame({
    "model": ["age + controls", "+ disease group", "+ physiology"],
    "age_OR": [np.exp(m.params["age"]) for m in (m1, m2, m3)],
    "pseudo_R2": [m.prsquared for m in (m1, m2, m3)],
    "log_likelihood": [m.llf for m in (m1, m2, m3)],
}).round({"age_OR": 3, "pseudo_R2": 4, "log_likelihood": 1})
triple
```

[10]:
```
              model  age_OR  pseudo_R2  log_likelihood
0   age + controls   1.015     0.0633         -5894.9
1  + disease group   1.023     0.1196         -5540.4
2     + physiology   1.024     0.1270         -5493.9
```

[11]:
```
# likelihood-ratio test: does physiology add signal beyond disease group (M3 vs␣
↪M2)?
lr_stat = 2 * (m3.llf - m2.llf)
```

```
lr_df = len(CORE_PHYS + SENS_PHYS)
lr_p = stats.chi2.sf(lr_stat, lr_df)
print(f"LR test (M3 vs M2): chi2 = {lr_stat:.1f}, df = {lr_df}, p = {lr_p:.2e}")
```

```
LR test (M3 vs M2): chi2 = 93.1, df = 5, p = 1.46e-18
```

3.1.2 The physiology odds ratios

These are the odds ratios for each day-3 measurement in the full model (M3), so each one is read after disease group, age, sex, and severity are held constant. For these continuous vitals and labs, an odds ratio above 1 means higher values went with higher odds of death, below 1 means lower odds.

[12]:
```
ci = m3.conf_int()
phys = CORE_PHYS + SENS_PHYS
phys_table = pd.DataFrame({
    "OR": np.exp(m3.params[phys]),
    "OR_ci_low": np.exp(ci.loc[phys, 0]),
    "OR_ci_high": np.exp(ci.loc[phys, 1]),
    "p_value": m3.pvalues[phys],
}).round(4)
phys_table
```

[12]:

	OR	OR_ci_low	OR_ci_high	p_value
meanbp	0.9962	0.9945	0.9979	0.0000
wblc	1.0045	0.9994	1.0096	0.0814
crea	1.0382	1.0098	1.0674	0.0081
alb	0.9518	0.8917	1.0159	0.1375
bili	1.0359	1.0243	1.0477	0.0000

[13]:
```
# VIF on the continuous predictors, flag anything above 5 as collinear
X = dmatrix("age + scoma + " + " + ".join(phys), model_df,␣
 ↪return_type="dataframe")
vif = pd.DataFrame({
    "predictor": [c for c in X.columns if c != "Intercept"],
    "VIF": [variance_inflation_factor(X.values, i)
            for i, c in enumerate(X.columns) if c != "Intercept"],
}).round(2)
vif
```

[13]:

	predictor	VIF
0	age	1.02
1	scoma	1.02
2	meanbp	1.02
3	wblc	1.03
4	crea	1.04
5	alb	1.04
6	bili	1.07

3.1.3 Sensitivity check on albumin and bilirubin

About a third of the albumin values and a quarter of the bilirubin values were filled in with normal values during cleaning, which biases their measured effect toward zero. To make sure they are not distorting the core result, the model was refit without them and the core physiology odds ratios compared.

[14]:
```
f3_core = f2 + " + " + " + ".join(CORE_PHYS)
m3_core = smf.logit(f3_core, model_df).fit(disp=False)
sens_compare = pd.DataFrame({
    "OR_with_alb_bili": [np.exp(m3.params[c]) for c in CORE_PHYS],
    "OR_without": [np.exp(m3_core.params[c]) for c in CORE_PHYS],
}, index=CORE_PHYS).round(4)
sens_compare
```

[14]:

	OR_with_alb_bili	OR_without
meanbp	0.9962	0.9958
wblc	1.0045	1.0055
crea	1.0382	1.0521

3.1.4 What the physiology added

Disease group carried the analysis. Pseudo R-squared went from 0.06 (age, sex, severity) to 0.12 once disease group was added, and only to 0.13 with physiology. The day-3 vitals and labs added only a small slice on top of disease group, though the likelihood-ratio test showed it was real (chi-square 93 on 5 df, p well below 0.001). Age stayed flat across the three models, so it held as a confounder.

Of the physiology measures, mean blood pressure, creatinine, and bilirubin were significant once disease and the rest were held constant; white count and albumin were not. Albumin coming out flat fits its heavy imputation with normal values. No collinearity showed up (all VIFs near 1), and dropping albumin and bilirubin barely moved the core odds ratios. Disease group still did most of the work. Physiology added a small but real increment, mostly from blood pressure, creatinine, and bilirubin.

4 ML-Q1: predicting 180-day mortality from day-3 data

RQ1 and RQ2 asked which factors are associated with death. ML-Q1 switches from explaining to predicting: given a patient's day-3 record, how well can a model flag who will not survive 180 days. It is judged on a held-out test set the model never saw, by how well it separates survivors from non-survivors (AUROC) and, because the use case is triage, by recall (how many of the patients who died it caught).

The features are the Gold model-ready set with two scrubs. sfdm2, a 2-month functional-status follow-up, and avtisst, the average TISS over days 3 to 25, were both dropped because they are recorded past the day-3 baseline and would leak future information into a day-3 prediction.

[15]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score, confusion_matrix, brier_score_loss,␣
 ↪roc_curve
from sklearn.preprocessing import StandardScaler
from sklearn.calibration import calibration_curve

gold = load_csv("support2_model_features.csv")
TARGET = "death_180d"
LEAK = ["sfdm2", "avtisst"]  # sfdm2 = 2-month follow-up; avtisst = avg TISS␣
 ↪over days 3-25, both reach past the day-3 baseline so they leak

y = gold[TARGET].to_numpy()
X = pd.get_dummies(gold.drop(columns=[TARGET] + LEAK, errors="ignore"),␣
 ↪drop_first=True).astype(float)
row_idx = np.arange(len(gold))

X_train, X_test, y_train, y_test, idx_train, idx_test = train_test_split(
    X, y, row_idx, test_size=0.30, stratify=y, random_state=42)
keep = X_train.columns[(X_train.std() > 1e-6) & ((X_train != 0).sum() >= 20)] ␣
 ↪# drop near-constant dummies
X_train, X_test = X_train[keep], X_test[keep]
print("train rows:", X_train.shape[0], " test rows:", X_test.shape[0], " test␣
 ↪deaths:", int(y_test.sum()))
```

```
train rows: 6373  test rows: 2732  test deaths: 1280
```

[16]:
```
import warnings

scaler = StandardScaler().fit(X_train)
forest = RandomForestClassifier(n_estimators=400, random_state=42, n_jobs=-1).
 ↪fit(X_train, y_train)
p_forest = forest.predict_proba(X_test)[:, 1]

# the logistic fit emits harmless numpy/BLAS matmul RuntimeWarnings on this␣
 ↪machine
# (the feature check found no degenerate column), so they are suppressed for␣
 ↪this fit
# only; the metrics below were checked and did not change.
with warnings.catch_warnings():
    warnings.simplefilter("ignore", RuntimeWarning)
    logit_clf = LogisticRegression(max_iter=5000).fit(scaler.
 ↪transform(X_train), y_train)
    p_logit = logit_clf.predict_proba(scaler.transform(X_test))[:, 1]

def scores(name, p):
    pred = (p >= 0.5).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, pred).ravel()
```

```
    return {"model": name, "AUROC": roc_auc_score(y_test, p),
            "recall": tp / (tp + fn), "specificity": tn / (tn + fp),
            "Brier": brier_score_loss(y_test, p)}

perf = pd.DataFrame([scores("logistic", p_logit),
                     scores("random forest", p_forest)]).round(3)
perf
```

[16]:

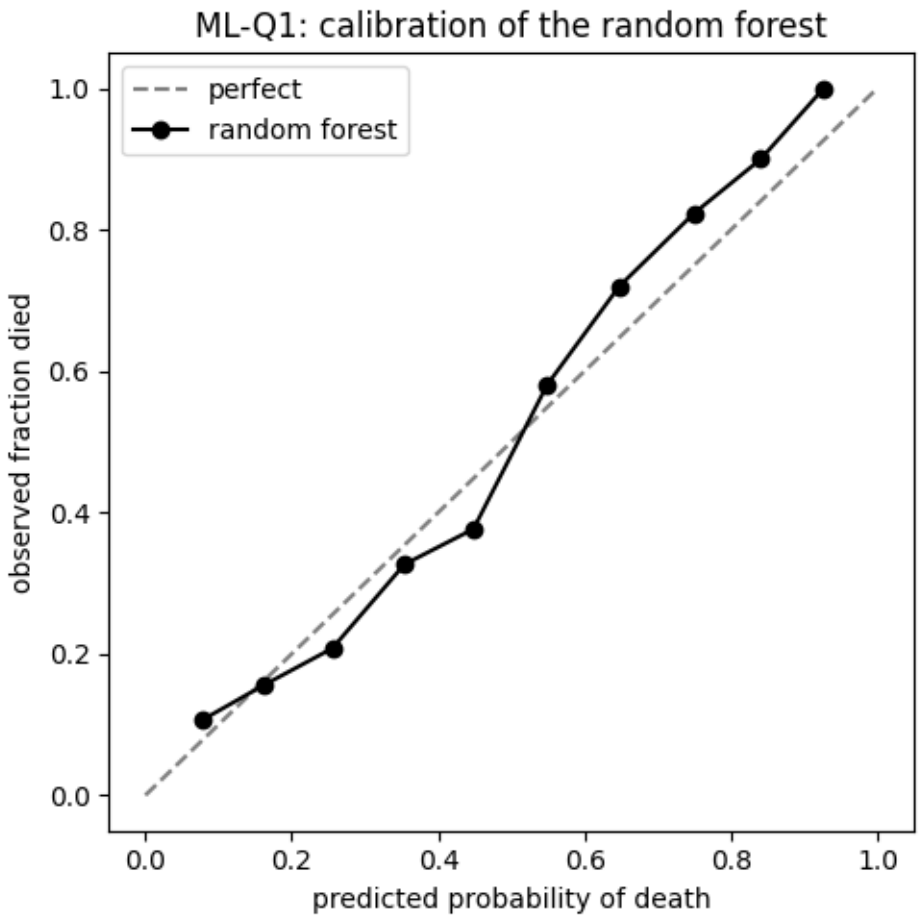
	model	AUROC	recall	specificity	Brier
0	logistic	0.745	0.598	0.764	0.203
1	random forest	0.761	0.638	0.767	0.199

[17]:
```
# confusion matrix for the random forest at the 0.5 threshold
cm = confusion_matrix(y_test, (p_forest >= 0.5).astype(int))
pd.DataFrame(cm, index=["actual survived", "actual died"],
             columns=["pred survived", "pred died"])
```

[17]:

	pred survived	pred died
actual survived	1113	339
actual died	463	817

[18]:
```
# calibration: do the predicted probabilities match the observed death rates?
frac_pos, mean_pred = calibration_curve(y_test, p_forest, n_bins=10)
fig, ax = plt.subplots(figsize=(5, 5))
ax.plot([0, 1], [0, 1], "--", color="gray", label="perfect")
ax.plot(mean_pred, frac_pos, "o-", color="black", label="random forest")
ax.set_xlabel("predicted probability of death")
ax.set_ylabel("observed fraction died")
ax.set_title("ML-Q1: calibration of the random forest")
ax.legend()
fig.tight_layout()
plt.show()
```



4.0.1 How well it predicted

On patients it had never seen, the random forest reached an AUROC of about 0.76 and the logistic about 0.75, above chance but not by a wide margin. Recall sat near 0.64, so the model caught a little under two thirds of the patients who actually died within 180 days. For triage that recall is the number that matters most, since a missed death is the costly error, and it is the first thing to push higher with threshold tuning. The calibration curve tracked the diagonal reasonably.

Two columns were dropped as leakage. With sfdm2 left in, both models scored near 0.90. Dropping sfdm2 brought it to about 0.81, but that still leaned on avtisst, the average TISS over days 3 to 25, which also reaches past the day-3 baseline. Dropping avtisst too gives the honest day-3 result, about 0.76.

5 ML-Q2: does the model beat the 1990s surv6m score?

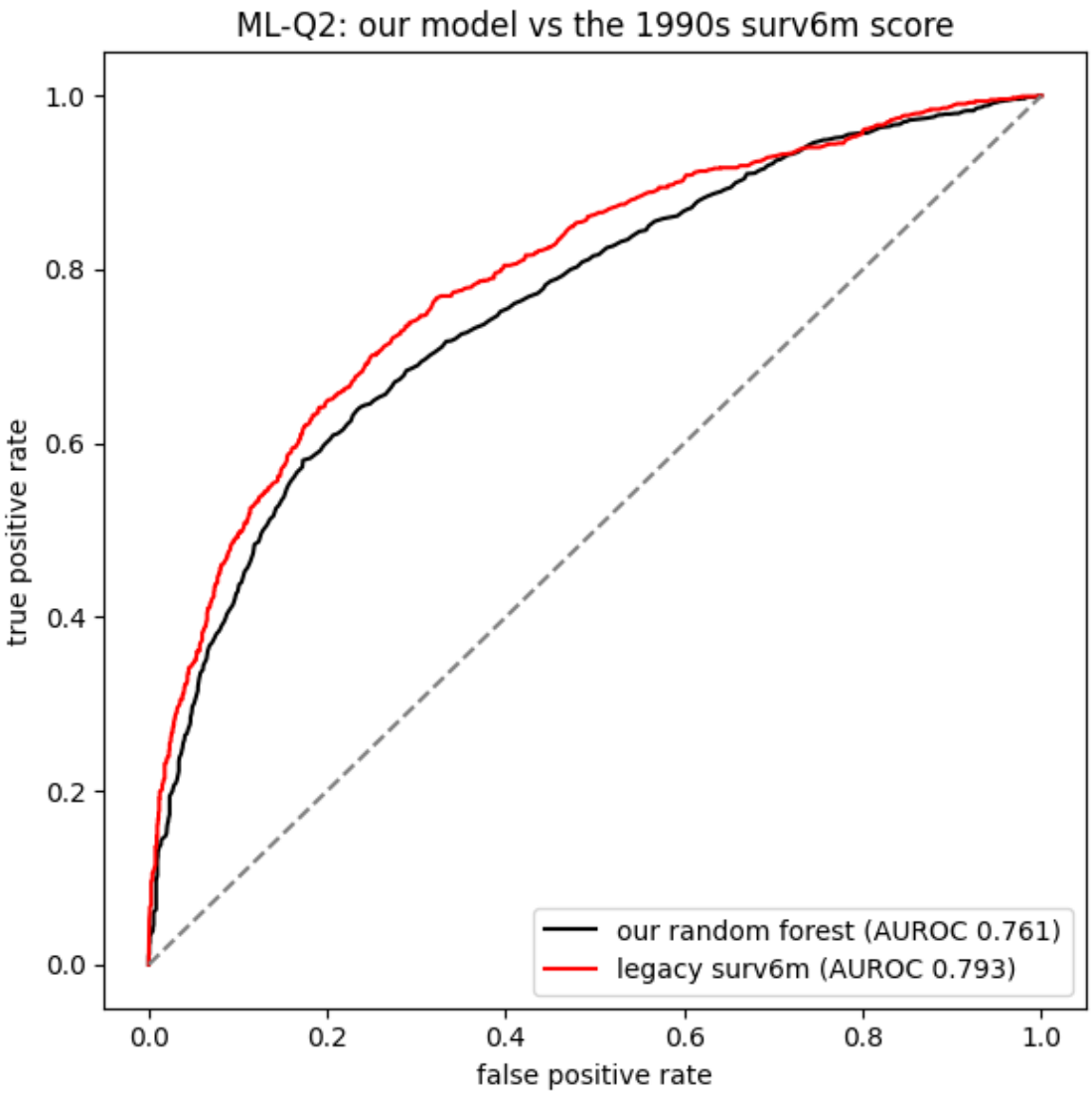
SUPPORT2 ships with surv6m, the 180-day survival estimate from the original 1990s SUPPORT model. ML-Q2 races our model against it on the same held-out test patients. surv6m is pulled from Silver and used only as the opponent, never as a feature inside our model, since it is a model

output and would leak.

[19]:
```
# surv6m is not in Gold (it is a leakage column), so it is read from Silver␣
 ↪here.
# Silver and Gold are the same rows in the same order, so idx_test aligns␣
 ↪positionally.
silver = load_csv("support2_cleaned.csv")
assert len(silver) == len(gold), "silver/gold row count mismatch"
assert (silver["age"].to_numpy() == gold["age"].to_numpy()).all(), "silver/gold␣
 ↪row order mismatch"

legacy = 1.0 - silver.iloc[idx_test]["surv6m"].to_numpy()  # old model␣
 ↪predicted 180-day death
assert len(legacy) == len(y_test), "legacy/test length mismatch"
auc_legacy = roc_auc_score(y_test, legacy)
auc_forest = roc_auc_score(y_test, p_forest)

fpr_f, tpr_f, _ = roc_curve(y_test, p_forest)
fpr_l, tpr_l, _ = roc_curve(y_test, legacy)
fig, ax = plt.subplots(figsize=(6, 6))
ax.plot(fpr_f, tpr_f, color="black", label=f"our random forest (AUROC␣
 ↪{auc_forest:.3f})")
ax.plot(fpr_l, tpr_l, color="red", label=f"legacy surv6m (AUROC {auc_legacy:.
 ↪3f})")
ax.plot([0, 1], [0, 1], "--", color="gray")
ax.set_xlabel("false positive rate")
ax.set_ylabel("true positive rate")
ax.set_title("ML-Q2: our model vs the 1990s surv6m score")
ax.legend(loc="lower right")
fig.tight_layout()
plt.show()
print(f"our random forest AUROC = {auc_forest:.3f}")
print(f"legacy surv6m AUROC     = {auc_legacy:.3f}")
```




```
our random forest AUROC = 0.761
legacy surv6m AUROC     = 0.793
```

5.0.1 Our model versus surv6m

Once the day-3 feature set was clean, the legacy score held up better than our model. The random forest came in at about 0.76 against surv6m's 0.79 on the same test patients, so the 1990s estimate edged our day-3 model by roughly 0.03 of AUROC. The earlier run that beat surv6m (0.81) was leaning on avtisst, which leaks. So the honest read flips: a model trained only on clean day-3 data did not beat the original SUPPORT estimate, it came in just under it. Matching it within a few points of AUROC from day-3 data alone is still a reasonable result, just not a win, and surv6m remains a strong baseline.

6 ML-Q3: do the same variables drive inference and prediction?

This ties the two halves together. The inference side (RQ1 and RQ2) ranked variables by how strongly they are associated with mortality (odds ratios). The prediction side ranks them by how much they drive the random forest (feature importance). ML-Q3 lays the two rankings side by side. Where they agree the finding is stronger; where they disagree, that is itself worth a look.

[20]:
```
importance = pd.Series(forest.feature_importances_, index=keep).
 ↪sort_values(ascending=False)
importance.head(10).round(4)
```

[20]:
```
age       0.0717
adlsc     0.0608
wblc      0.0563
scoma     0.0553
meanbp    0.0552
hrt       0.0523
temp      0.0499
crea      0.0494
pafi      0.0474
sod       0.0467
dtype: float64
```

[21]:
```
# the shared day-3 physiology variables: inference (RQ2) vs prediction␣
 ↪(importance)
compare_vars = ["meanbp", "wblc", "crea", "alb", "bili"]
bridge = pd.DataFrame({
    "inference_OR": [phys_table.loc[v, "OR"] for v in compare_vars],
    "inference_p": [phys_table.loc[v, "p_value"] for v in compare_vars],
    "rf_importance": [float(importance.get(v, float("nan"))) for v in␣
 ↪compare_vars],
}, index=compare_vars).round(4)
bridge
```

[21]:

	inference_OR	inference_p	rf_importance
meanbp	0.9962	0.0000	0.0552
wblc	1.0045	0.0814	0.0563
crea	1.0382	0.0081	0.0494
alb	0.9518	0.1375	0.0417
bili	1.0359	0.0000	0.0462

6.0.1 Where the two views line up

The two rankings agreed only in part. With avtisst removed as a leak, the prediction side spread its importance across age, the ADL score (adlsc), white count, coma score, mean blood pressure, and a broad set of day-3 vitals, rather than leaning on any single dominant variable. The inference side put disease group out front (RQ1), with physiology adding a small increment (RQ2). Both point at overall severity and physiology mattering, but disease group, which carried the strongest

inference signal, did not dominate the prediction importance.

Among the shared physiology variables the overlap was loose: some that were significant in the inference model carried little random-forest importance, and one that was not significant ranked higher, so the two lenses do not map one to one. Severity and physiology matter in both views, but no single variable runs the table in both. An odds ratio and a feature importance answer different questions, and that is the reason not to treat them as the same number.